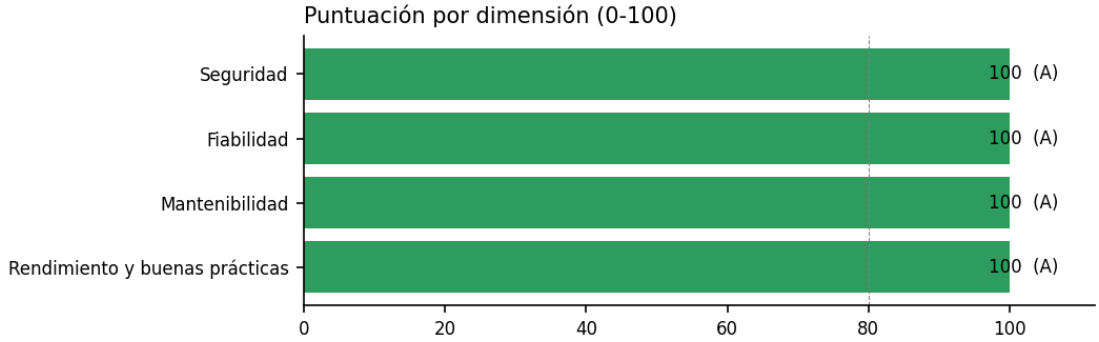
Informe de análisis estático de código

Solución StarWarsWeb · rama develop · 30/10/2026

Calificación general de calidad

A

100.0 / 100 - Excelente
1.1 KLOC de producción analizadas (sin tests)



Dimensión	Peso	Puntos ponderados	Puntos / KLOC	Puntuación	Nota
Seguridad	30%	0	0.0	100	A
Fiabilidad	30%	0	0.0	100	A
Mantenibilidad	25%	0	0.0	100	A
Rendimiento y buenas prácticas	15%	0	0.0	100	A

Cómo se calcula: cada hallazgo suma puntos (patrón Alta = 5.0, Media = 2.0, Baja = 0.25; advertencias de analizador = 0,5 a 1; método largo o complejo = 1). Los puntos se dividen entre las KLOC de producción y la puntuación es 100 - 100 x (puntos/KLOC) / 10, con mínimo 0. La nota general es el promedio ponderado (Seguridad 30 %, Fiabilidad 30 %, Mantenibilidad 25 %, Rendimiento 15 %). Escala: A >= 90, B >= 80, C >= 65, D >= 50, E < 50. Es un indicador relativo y heurístico, útil para comparar entre ejecuciones; no sustituye una revisión de código.

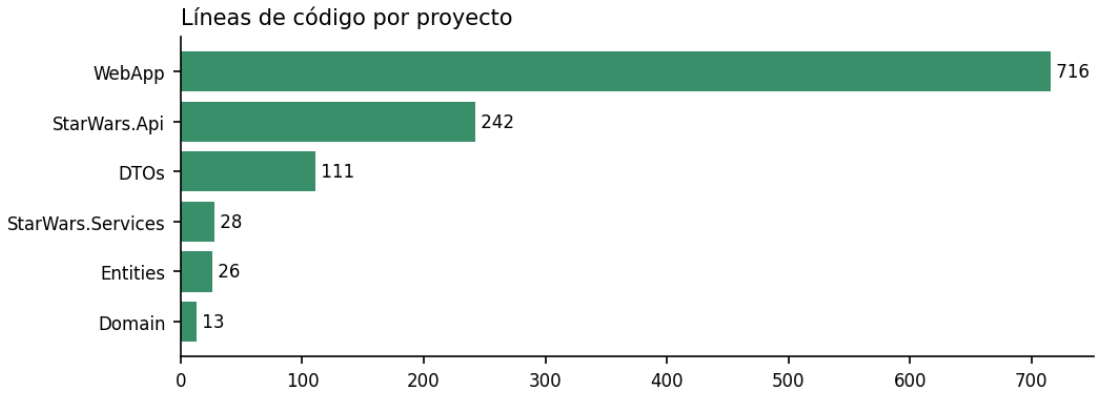
1. Resumen ejecutivo

Se analizaron **28** archivos (.cs y .razor, excluyendo bin/obj/Migrations) con **1,136** líneas no vacías en 6 proyectos. El análisis combinó (a) el compilador de .NET con analizadores Roslyn (AnalysisMode=Recommended) sobre las 5 APIs y la WebApp, (b) reglas de patrones propias en Python y (c) métricas de tamaño y complejidad ciclomática aproximada.

Indicador	Valor
Advertencias de compilador/analizadores	0
de ellas, nulabilidad (CS86xx)	0
Hallazgos de patrones - severidad Alta	0
Hallazgos de patrones - severidad Media	0
Hallazgos de patrones - severidad Baja	0
Métodos con más de 60 líneas	0
Métodos con complejidad >= 15	0
Controladores sin [Authorize] (de 3)	3
Controladores con acceso directo a DbContext	0
Archivos con using duplicados	0

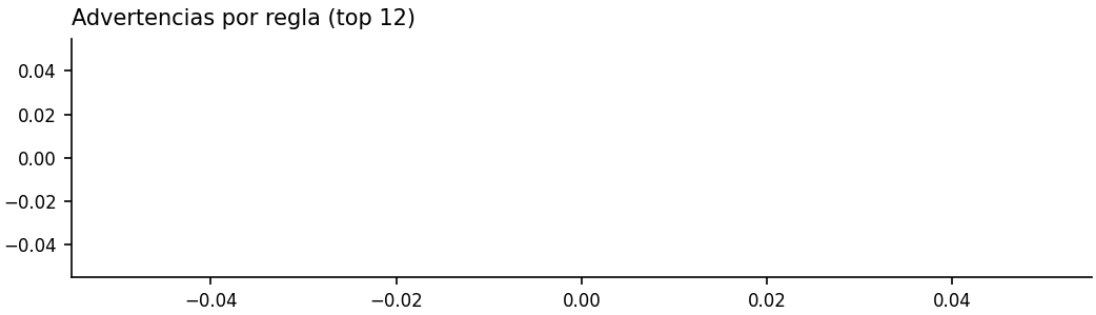
Alcance y limitaciones: MovilApp y DesktopApp no se compilaron (requieren cargas de trabajo Android/Windows no instaladas); sólo se les aplicaron patrones y métricas. La complejidad y longitud de métodos son aproximaciones léxicas, no un AST completo. Los hallazgos son indicios que requieren revisión humana.

2. Tamaño de la solución



Proyecto	Archivos	Líneas	Líneas de comentario
WebApp	15	716	23
StarWars.Api	4	242	18
DTOs	5	111	3
StarWars.Services	1	28	4
Entities	2	26	3
Domain	1	13	0

3. Compilador y analizadores Roslyn



Regla	Categoría	Cantidad	Descripción

Por proyecto

Proyecto	Advertencias

Archivos con más advertencias

Archivo	Advertencias

4. Hallazgos por patrones

Severidad	Categoría	Regla	Ocurrencias

5. Arquitectura y controladores

De 3 controladores, **3** no declaran [Authorize] en el archivo (puede haber autorización global o por política; verificar) y **0** parecen usar el contexto de datos directamente, contraviniendo la convención del proyecto de mantener la lógica en servicios.

Sin [Authorize]
2.- Backend/2.1- Microservices/StarWars.Api/Controllers/AuthController.cs
2.- Backend/2.1- Microservices/StarWars.Api/Controllers/CharacterController.cs
2.- Backend/2.1- Microservices/StarWars.Api/Controllers/FilesManagementController.cs

6. Métodos largos y complejos

Archivo	Método	Línea	Líneas	CC

Mayor complejidad ciclomática (aprox.)

Archivo	Método	Línea	Líneas	CC

7. Recomendaciones priorizadas

1. Confirmar la política de autorización de los controladores sin [Authorize] y definir una política global por defecto.

2. Integrar el análisis en CI con un .editorconfig compartido y un umbral para nuevas advertencias.